

Tutorium 11

Suchalgorithmen

Grundlagen der Praktischen Informatik

15. Juni 2026

Suchen in Listen

Praxis

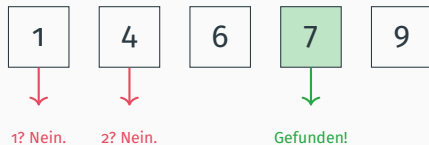
Die einfachste Methode: Wir überprüfen jedes Element der Liste nacheinander.

Lineare Suche in Haskell

```
1 contains :: Eq a => [a] -> a -> Bool
2 contains [] _ = False
3 contains (y:ys) x
4     | x == y      = True
5     | otherwise = contains ys x
```

- ▶ **Laufzeit:** $\mathcal{O}(n)$
- ▶ Wir müssen im *Worst Case* jedes Element genau einmal betrachten.
- ▶ Rekurrenzgleichung: $T(n) = T(n - 1) + \mathcal{O}(1)$

Lineare Suche (Visuell)



Im **Worst Case** müssen wir die komplette Liste ablaufen.

Wenn die Liste **bereits sortiert** ist, können wir viel effizienter vorgehen: Wir halbieren den Suchraum bei jedem Schritt!

Binäre Suche in Haskell

```
1 containsBinary :: Ord a => [a] -> a -> Bool
2 containsBinary xs x = go xs
3   where
4     go [] = False
5     go ys = let mid = length ys `div` 2
6               midVal = ys !! mid
7               in case compare x midVal of
8                 LT -> go (take mid ys)
9                 GT -> go (drop (mid + 1) ys)
10                EQ -> True
```

Binäre Suche (Visuell)

Suche nach **7**:

1	4	6	7	9	10	213	534	865	999
---	---	---	---	---	----	-----	-----	-----	-----



1	4	6	7	9
---	---	---	---	---

Rechte Hälfte verworfen)



6	7	9
---	---	---

Bei jedem Schritt halbieren wir den Suchraum! $\rightarrow \mathcal{O}(\log n)$

- ▶ **Laufzeit:** $\mathcal{O}(\log_2 n)$
- ▶ Da die Liste in jedem Schritt halbiert wird, entspricht die Anzahl der Schritte der Höhe eines Binärbaums.
- ▶ **Beispiel:** Bei einer Million Einträge benötigen wir **maximal 20 Vergleiche!**

[1, 4, 6, 7, 9, 10, 213, 534, 865, 999]

Wir vergleichen in der Mitte und werfen jeweils die unpassende Hälfte weg.

Live Demo



05_Suchen.txt

Gemeinsame Analyse und Programmierung der Suchalgorithmen.